

Shakespeare AI: Scholar Studio

Official Competition Submission & Technical Description Document

Abstract: Shakespeare AI is a multi-agent scholar companion designed to bridge the gap between four centuries of classical literature and modern current events. Grounded in the philosophy that **we are doomed to repeat history unless we study its patterns in the present**, this project utilizes a graph-based multi-agent system (ADK 2.0) to analyze contemporary news, write contextual comparative essays, and navigate Shakespeare's complete works. Built using a decoupled local-first SQLite architecture, semantic safety guardrails, and a dual-step search grounding cache refresher, this application demonstrates innovative tool use, absolute session isolation, and extreme token efficiency.

1. Central Idea and Innovation

The core idea of Shakespeare AI is to make classical literature active, relevant, and conversational by pairing it with modern real-world events. Instead of treating Shakespeare's plays and sonnets as static historical artifacts, the platform uses them as a critical mirror to reflect on current global issues, environmental crises, and human behaviors. The platform's innovation lies in its **multi-agent cooperative intelligence**, where rather than querying a single large language model (LLM) with a monolithic prompt, the system divides labor among a specialized network of ten distinct agents. This architecture ensures high-fidelity queries, strict user privacy, and contextual accuracy.

2. Problem Statement

Scholars and students often struggle to find relevance in classical literature, viewing it as distant from their modern realities. However, human struggles—political ambition, betrayals, societal breakdowns, and environmental distress—have not changed. We are bound to repeat history unless we learn to recognize its patterns in our present day. Shakespeare AI solves this by matching real-time news and climate stories with highly applicable Shakespearean dialogue and providing critical commentary. This connection encourages deeper literary engagement and historical self-reflection, making classical education highly accessible, interactive, and impactful.

3. Why Agents? The Multi-Agent Ecosystem

A single LLM lacks the focus, safety guardrails, and deterministic tool behaviors required for high-quality academic research. To address this, the project deploys a modular, multi-agent system composed of ten distinct, cooperative nodes:

- **Security Guardrail Agent:** A pre-flight filter that analyzes user queries for prompt injection, system bypass, or model hijacking attempts, blocking malicious requests before they consume downstream resources.
- **Intent Routing Agent (ADK Router):** Classified by a strict Pydantic output schema, this agent analyzes the user's message and routes it to the single most relevant subagent, preventing prompt bloat and context drift.
- **Corpus Reader Agent:** Retrieves raw text dialogue from the local database using structured queries.
- **Corpus Searcher Agent:** Instantly crawls the SQLite database for occurrences of words, characters, or phrases.
- **History Scholar Agent:** Specialized in Shakespeare's biography and historical context, reading local Markdown facts files.
- **Scholarship Agent:** Connects to external search APIs to fetch modern academic papers and literary criticism.
- **Advice Agent:** Formulates sayings, quotes, and wisdom tailored to user-described modern dilemmas.
- **General Chat Agent:** Main conversational interface representing the friendly and serene persona of the Bard.
- **Current Events Agent:** A background worker that uses Google Search Grounding to pull current global headlines.
- **News Analysis Agent:** Synthesizes news data and writes multi-paragraph philosophical commentary.

4. Overall Architecture

The system is built on a decoupled, client-server model. The frontend is a responsive, single-page application written in HTML5, CSS3, and JavaScript, featuring smooth transition navigation and a mobile-optimized dual-dropdown works selector. The backend is powered by FastAPI, serving as the bridge to the SQLite database and the multi-agent graph.

A custom **X-Session-ID** header generated by the client is passed with every API request. This UUID partitions user histories, local SQLite telemetry logs (capturing IP addresses, locations, and debounced scroll markers), and telemetry traces, ensuring complete session isolation and eliminating the risk of session bleeding in stateless environments like Google Cloud Run.

5. Clever Tool Use and Technical Highlights

The codebase features several advanced engineering patterns designed for stability and credit management:

- **Dual-Step Grounding Pattern:** To enforce JSON formatting schema constraints during web searches, the system uses a dual-step pattern. First, the model is called with the Google Search tool enabled to fetch citation-rich web contexts. Second, the raw output is sent to a structured schema generator with tools disabled to parse it into a strict Pydantic object. This isolates information discovery from schema enforcement, ensuring 100% data contract compliance.
- **Age-Based Cache Validation:** To prevent credit drain on Cloud Run, the system runs an age-based cache check. Before executing expensive search grounding API loops, it validates the modification time of local JSON caches. If the file is fresh, it serves the cached data immediately. This results in extreme token conservation.
- **Self-Healing Startup Bootstrap:** During startup, the FastAPI server validates the SQLite database's structure. If the database is missing or blank (often due to gitignores in container builds), the server automatically downloads the clean database and reconstructs the historical files, providing a seamless zero-configuration setup for developers.

6. Project Development Journey

The development of Shakespeare AI was a collaborative pair-programming journey between the developer and the **Antigravity** coding agent. The project evolved through several iteration phases: 1) *Database Setup*: Implementing `setup_data.py` to pull Catherine Devlin's open-source database and constructing the SQLite schema; 2) *Responsive Styling*: Resolving viewport locking and scroll-container freezes on mobile devices by defining flexbox boundary overrides in `index.css`; 3) *Asset Refinement*: Engineering a background removal script that scrubbed checkerboard pixels from the Shakespeare image to enable a clean, bouncing Max Headroom animation; 4) *Test Automation*: Writing a comprehensive pytest suite to run unit and integration checks, resolving ADK session name conflicts.

7. Live Deployment & Reproduction Guide

The application is fully containerized and deployed to **Google Cloud Run**, configured to scale down to 0 instances when idle to eliminate billing overhead. To reproduce this deployment:

```
# 1. Authenticate with Google Cloud
gcloud auth login
gcloud config set project [YOUR_PROJECT_ID]

# 2. Build and Deploy using Agents CLI (which wraps gcloud run deploy)
agents-cli deploy \
  --project [YOUR_PROJECT_ID] \
  --region us-east1 \
  --secrets GEMINI_API_KEY=gemini-api-key:latest \
  --no-confirm-project
```

3. Restrict idle billing by updating min-instances to 0

```
gcloud run services update shakespeare \
```

```
--min-instances 0 \
```

```
--region us-east1 \
```

```
--project [YOUR_PROJECT_ID]
```

4. Grant public access to the invoker role

```
gcloud run services add-iam-policy-binding shakespeare \
```

```
--member="allUsers" \
```

```
--role="roles/run.invoker" \
```

```
--region=us-east1 \
```

```
--project=[YOUR_PROJECT_ID]
```